

MC521 - Relatório II

Júlio da Silva Telles

16 de Julho de 2026

1 D - Asya and Kittens (CodeForces - 1131F)

O problema D descreve uma situação em que temos n gaiolas numeradas de $1 - n$ lado a lado cada uma contendo um gato. Em um certo momento pares de gatos em celas adjacentes podem formar pares, então remove-se a divisão entre suas gaiolas para que possam brincar, até que todos os gatos estejam na mesma gaiola. Dado apenas a ordem em que as uniões de gaiolas foram estabelecidas o problema pede a ordem inicial em que as gaiolas estevam, antes da junção dos pares. A imagem abaixo contem um dos varios possiveis arranjos iniciais de gaiolas para o seguinte caso teste: $(1, 4), (2, 5), (3, 1), (4, 5)$

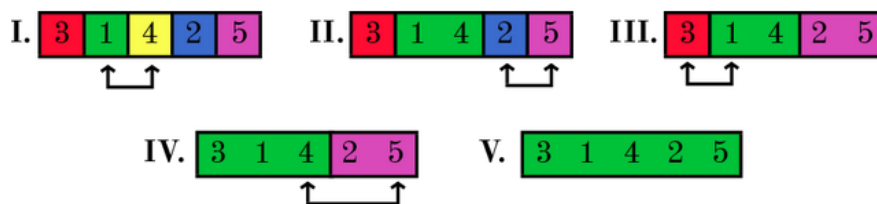


Figura 1: Legenda descritiva da sua imagem.

1.1 Solução

Vamos utilizar um algoritmo de DSU para resolver esse problema. O algoritmo realiza a união de conjuntos dinâmicos de forma eficiente e registra o histórico dessas conexões. O objetivo principal é imprimir a sequência unificada de elementos na ordem inversa: partindo da **folha** (fim da fusão) até a **raiz** (início da fusão). Para isso, gerenciam-se duas estruturas de dados simultâneas: uma árvore para os conjuntos e uma lista encadeada para a ordenação.

- Inicialização: Cada elemento começa isolado em seu próprio conjunto, sendo simultaneamente a **cabeça** (início) e a **cauda** (fim) de sua respectiva lista encadeada.
- Operação de União (**unionSet**): Ao unir dois elementos, o algoritmo executa dois passos independentes:
 - **Otimização do DSU (Árvore):** Utiliza a heurística de *rank*, conectando a raiz da árvore menor à raiz da árvore maior. Isso garante que a altura da árvore permaneça mínima, mantendo a complexidade de busca quase constante $\mathcal{O}(\alpha(N))$.

- **Encadeamento das Listas (Ordem):** Conecta o fim (cauda) da lista do primeiro conjunto ao início (cabeça) da lista do segundo conjunto. A cauda do conjunto resultante é atualizada para ser a cauda do segundo grupo.

Após todas as uniões e retornamos a ordem específica de impressão guardada na lista encadeada.

2 F - Roads not only in Berland (CodeForces - 25D)

No problema F o prefeito da cidade de Berland busca conectar Berland a todas as cidades proximas, por meio da construção de estradas novas conectando Berland às demais, no lugar de estradas velhas que são desnecessaria. Nesse problema e dado um grafo não direcionado não conexo no qual o objetivo é criar o minimo de arestas necessárias para conectar a componente contendo o vertice 1, Berland. No entanto a cada nova aresta construida é necessário eliminar uma aresta antiga que não afete o grau de conectividade do grafo. A saída esperada é composta pelo numero minimo de arestas novas necessárias para que o grafo se torne conexo, e o plano para que isso ocorra, a cada linha de aresta nova que precisa ser feita e necessario imprimir os vertices que compõem essa aresta, e uma aresta que foi removida.

2.1 Solução

Vamos utilizar um algoritmo de DSU para resolver o problema. Tome inicalmente cada vertice do grafo como um conjunto unitário, e a cada aresta nova adicionada nesse grafo realizamos a união desses dois conjuntos (vertices) chamando o metodo `unionSet()`, caso esses dois vertices estejam no mesmo conjunto (componente) a aresta é colocada em um `Vector<pair<int, int>` de arestas não necessárias. Após a contrução inicial das componentes e da lista de arestas descartaveis, a resposta será o número de novas arestas, que será igual ao numero de conjuntos disjuntos -1 (conjunto que contém o vértice 1). Posteriormente fazemos um laço sobre todos os vértices do grafo, e verificamos se eles estão conectados ao conjunto contendo o vertice 1, caso contrario unimos ele à componente do vertice 1 e então imprimimos a nova aresta seguida de uma das arestas não utilizadas e em seguida removemos essa aresta não utilizada da lista. A função abaixo implementa a logica de conexão de arestas novas e remoção de arestas descartaveis.

```
1 void printRoads(){
2     int i = 2;
3     while (sz(unUsed) != 0){
4         if (!isSameSet(1, i)){
5             unionSet(1, i);
6             ii old_road = unUsed[sz(unUsed) - 1];
7             unUsed.PP();
8             cout << old_road.first << ' ' << old_road.second <<
              ' ' << 1 << ' ' << i << '\n';
9         }
10        i++;
11    }
12 }
```